Doc. No.:	WG21/P1217R2
Date:	2019-06-16
Reply-to:	Hans-J. Boehm
Email:	hboehm@google.com
Authors:	Hans-J. Boehm, with input from many others
Audience:	SG1

P1217R2: Out-of-thin-air, revisited, again

Abstract

We propose to add an additional memory_order to C++, which is essentially a slightly stronger version of `memory_order_relaxed` that is amenable to precise semantic definition. It avoids certain anomalies of `memory_order_relaxed` that scare some of us.

This began as a status update attempting to summarize an external discussion of so-called out-of-thin-air results with `memory_order_relaxed`. It was known that allowing such results often makes it impossible to reason about code using `memory_order_relaxed`, and that our current wording prohibiting them is excessively vague. Many of us believed that this was a stopgap until we determined a better way to word this restriction without invalidating current implementations. It has become much more clear that this cannot happen in a way that would satisfy all of us; current implementations in fact allow results that, unless we redefine the semantics of basic sequential constructs like if-statements, can only be understood as something hard-to-distinguish from out-of-thin-air results.

Acknowledgements

This benefited greatly from discussions and debates with many others, including Sarita Adve, Will Deacon, Brian Demsky, Doug Lea, Daniel Lustig, Paul McKenney, and Matt Sinclair. I believe we all agree that there is a problem that should be addressed, but not on a detailed solution.

History

This topic was briefly discussed by SG1 at the Rapperswil meeting. This paper is in part an attempt to respond to the question, raised at that meeting, as to whether we can just live with the results allowed by current implementations.

The paper was discussed in San Diego, without a clear consensus how to proceed.

R1 corrects a mistake in the "data_wrapper" example that was pointed out by Derek Dreyer.

R2 replaced the long "Effect on sequential reasoning ..." section with a much shorter example that I would like to see included in the standard. The old section had a more thorough discussion of invalidating sequential reasoning for modules only accessed by a single thread, and may still be useful for that. However the new "Another scary example" section largely makes the same points. It adds sections on prior discussion and suggests a compromise

proposal. It also raises a question about whether the function calls in OOTA Example 3 are even necessary, along with making several other minor changes and corrections.

The out-of-thin-air problem

The out-of-thin-air problem has been repeatedly discussed, e.g. in N3710, and http://plv.mpi-sws.org/scfix/paper.pdf.

The fundamental problem is that without an explicit prohibition of such results, the C++ memory model allows `memory_order_relaxed` operations to execute in a way that introduces causal cycles, making it appear that values appeared out of thin air, commonly abbreviated as OOTA. The canonical example is the following, where `x` and `y` are atomic integers initialized to zero:

OOTA Example 1	
Thread 1	Thread 2
`r1 = x.load(memory_order_relaxed);` `y.store(r1, memory_order_relaxed);`	`r2 = y.load(memory_order_relaxed);` `x.store(r2, memory_order_relaxed);`

Without specific requirements to the contrary, the loads are allowed to see the effects of the racing stores. Thus an execution in which the stores write some arbitrary value, say 42, and the loads read those values, is valid. This can be interpreted as an execution in which both loads speculatively return 42, then perform the stores (e.g. to memory only visible to these two threads, where it could be rolled back if we guessed wrong), and then check that the guesses for the loads were correct, which they now are.

Thus with OOTA execution allowed, the above code can set `x` and `y` to a value that is computed nowhere in the code, and can only be justified by a circular argument, or one involving explicit reasoning about speculation.

Similarly, consider the following program in a world with out-of-thin-air results allowed:

OOTA Example 2	
Thread 1	Thread 2

`if (x.load(memory_order_relaxed))`
`  y.store(1,`
`memory_order_relaxed);`
	`if (y.load(memory_order_relaxed))`
	`  x.store(1,`
	`memory_order_relaxed);`

Again both stores can be speculated to occur, and be read by the loads, thus validating the speculation, and causing both variables to be set to 1. This is annoying because the corresponding program written without atomics, is data-race-free, and always does nothing, as expected.

However the worst scenario in this OOTA world are arguably cases in which OOTA loads result in precondition violations for some opaque third-part library:

Assume that `x` and `y` are declared as integers, but are only ever assigned 0, 1, or 2, and that functions `f()` and `g()` take one such 0-to-2 argument each. `f()` and `g()`'s precondition effectively includes the restriction that its argument is between 0 and 2.

OOTA Example 3	
Thread 1	Thread 2
`r1 = x.load(memory_order_relaxed))` `f(r1)`	`r2 = y.load(memory_order_relaxed))` `g(r2);`

Again, we have to consider an execution in which each load guesses that the value seen is e.g. 42. This then gets passed to `f()` and `g()`, which are allowed to do anything, since their precondition is not satisfied. "Anything" includes assigning 42 to `y` and `x` respectively, e.g. as a result of an out-of-bounds array reference. This action would again validate the speculation, making it legal.

This last example is particularly disconcerting, since it seems essentially impossible to for the programmer to avoid in real code. It only requires that we perform relaxed loads from two variables in two different threads, and pass the result to some functions that make some assumption about their input. Any significant code performing relaxed loads on pointers, and relying on the result being a valid pointer, is essentially certain to contain a variant of this pattern.

It is not even completely clear that the function calls in the last example are necessary. Can the loads themselves produce a trap representation, so that assignment to the local "register" values causes undefined behavior, thus causing writes of the trap values to the shared variables?

Fortunately nobody believes that any of the above three examples can actually produce these results in real implementations. All common hardware disallows such results by prohibiting a store from speculatively executing before a load on which it "depends". Unfortunately, as we will see below, the hardware notion of "depends" is routine invalidated by many important compiler transformations, and hence this definition does not carry over to compiled programming languages.

Although such results are not believed to occur in practice, the inability to precisely preclude them has serious negative effects. We have no usable and precise rules for reasoning about programs that use `memory_order_relaxed`. This means we have no hope of formally verifying most code that uses `memory_order_relaxed` (or `memory_order_consume`). "Formal verification" includes partial verification to show the absence of certain exploits. Since we don't have a precise semantics to base it on, informal reasoning about `memory_order_relaxed` also becomes harder. Compiler optimization and hardware rules are unclear.

It is also worth noticing that so far we have constrained ourselves to examples in which each thread is essentially limited to two lines of code. The effect on more realistic code is not completely clear. Most of this paper focusses on a new class of examples that we hadn't previously explored.

The status quo

However, we have been unable to provide a meaningful C++-level definition of "out-of-thin-air" results. And thus the standard has not been able to meaningfully prohibit them. The current standard states:

> “Implementations should ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation.”

while offering only examples as to what that means. (See N3786 for a bit more on how we got here. This did not go over well in WG14.)

So far, our hope has been that this problem could be addressed with more ingenuity on our part, by finally developing a proper specification for "out-of-thin-air" results. Here I explain why I no longer believe this is a reasonable expectation.

What changed

Although it was previously known (see e.g. Boehm and Demsky and Bill Pugh et al's much earlier Java memory model litmus tests) that there were borderline cases, which had a lot of similarity to out-of-thin-air results, but could be generated by existing implementations, it wasn't fully clear to us how similar these can get to true out-of-thin-air results, and the catastrophic damage they can already do. I no longer believe that it is possible to draw a meaningful line between these and "true" out-of-thin-air results. Even if we could draw such a line, it would be meaningless, we would still have no good way to reason about code on the "right" side of the line since, as we show below, that can still produce completely unacceptable results.

Note that we still do not have actual code failures that have been traced to this problem; we do now have small sections of contrived code that can result in the problems discussed here. And we do not have any way to convince ourselves that real code is immune from such problems. The most problematic cases are likely to involve atomics managed by different modules, with the atomic accesses potentially quite far apart. Thus failure rates are likely to be very low, even if the problem does occur in real code.

The central problem continues to be our inability to reason about code using `memory_order_relaxed`.

A new class of out-of-thin-air results

Unfortunately, our initial canonical out-of-thin-air example above can be turned into a slightly more complicated example that can generate the same questionable result. Abstractly, as defined by the standard, the only difference between the two examples is an unexecuted conditional, something that should not make any difference. And I believe it cannot make any difference with anything like the specification we use in the current standard.

In external discussion, such examples have been dubbed "RFUB" ("read from unexecuted branch") instead of OOTA. Our initial example, with additions over OOTA Example 1 highlighted, is:

RFUB Example 1	
Thread 1	Thread 2

`r1 = x.load(memory_order_relaxed);`
`y.store(r1, memory_order_relaxed);`
	`bool assigned_42(false);`
	`r2 = y.load(memory_order_relaxed);`
	`if (r2 != 42) {`
	`  assigned_42 = true;`
	`  r2 = 42;`
	`}`

	`x.store(r2, memory_order_relaxed);` `assert_not(assigned_42);`

We argue that entirely conventional optimizations can result in an execution *in which the assertion succeeds*, but `x` and `y` have been assigned 42. In reality, this is achieved with a combination of hardware and compiler transformations.

(We assume that `assert_not` is a separately compiled function, with the implied semantics. In our environment, we get different code if we use the actual `assert` macro.)

The compiler transformations proceed roughly as follows:

1. Notice that the store to `x` must always assign 42. Update it to just store 42.
2. The assignment to `r2` is now dead. Remove it.
3. Replace the remaining conditional, whose body now just assigns to `assigned_42`, with essentially `assigned_42 = (r2 != 42)`.

This gives us essentially:

```
bool assigned_42;
r2 = y.load(memory_order_relaxed);
assigned_42 = (r2 != 42);
x.store(42, memory_order_relaxed);
assert_not(assigned_42);
```

Gcc 7.2 -O2 on Aarch64 (ARMv8) generates (from the source in the table):

```
ldr     w1, [x0]   // w1 = y.load(...)
add     x0, x0, 8  // x0 = &x
mov     w2, 42
str     w2, [x0]   // x = 42
cmp     w1, w2     // w0 = (y != 42)
cset    w0, ne
b       _Z10assert_notb  // call assert_not(w0)
```

Since ARMv8 allows an independent load followed by an independent store to be reordered, the store (`str`) instruction may be executed before any of the other instructions, effectively transforming the code to:

```
x.store(r2, memory_order_relaxed);
bool assigned_42;
r2 = y.load(memory_order_relaxed);
assigned_42 = (r2 != 42);
assert_not(assigned_42);
```

If Thread 1 executes immediately after the store instruction, before any other Thread 2 instructions, the load of `y` will read 42, `assigned_42` will be false, and we will end up with exactly the problematic execution.

Note that there is no real obstacle to this occurring on a strongly ordered architecture like x86 as well; the hardware would not reorder a load with a later store, but there is no rule preventing the compiler from doing so. It is just difficult to construct examples in which that would be a profitable compiler transformation. On the other hand, the hardware may find this to be profitable if e.g. the load misses the cache.

This behavior makes it look as though the then-clause in the condition was partially executed. That is clearly not allowed by the current spec. But, aside from the vague "normative encouragement" to avoid OOTA results, this can be explained as the loads at the beginning of both threads speculatively observing the stores at the end, i.e. as an OOTA result.

Unfortunately, this means that current mainstream implementations are not actually following the "normative encouragement" to avoid OOTA results.

The separate external discussion has considered a number of RFUB examples. They don't seem quite as disastrous as OOTA Example 3 above. But the following section argues that reasoning about programs with RFUB results remains essentially intractable, since it invalidates arguments about properly used sequential code.

Another Scary example

Consider the following "RFUB New Constructor" example. Assume `Foo` is a class type, with virtual member function `f()`, and we have the following declaration:

```
atomic<Foo *> x(nullptr), y(nullptr);
```

Then consider a program consisting of the following two threads:

RFUB New Constructor Example	
Thread 1	Thread 2

`r1 = x.load(memory_order_relaxed);`
`y.store(r1, memory_order_relaxed);`
	`Foo* r2 = y.load(memory_order_relaxed);`
	`if (r2 == nullptr) {`
	`  r2 = new Foo();`
	`}`
	`x.store(r2, memory_order_relaxed);`
	`r2->f(); // UB`

In the absence of our current vague OOTA discouragement, again each load is allowed to see the other thread's store. Thread 2 will always store a pointer to a newly allocated object, so Thread 1's load may see that store. We obtain a consistent execution if Thread 1 stores that same pointer, and Thread 2 loads it. But in this execution, the `new` expression in Thread 2 is never executed! Thus the object whose address ends up in r2 is never constructed!

This execution is not likely with current implementations, but it is entirely plausible if we make favorable assumptions about the `operator new` implementation:

Assume that operator new allocates from a region dedicated to the thread calling it. Depending on the details, it may be possible to determine, using conventional flow analysis, that the new expression in Thread 2 can only

return the first location in Thread 2's buffer. Call that location *q*. Although *q* might not be a constant, it can be easily computed ahead of time, without consulting any of the program variables.

A standard points-to analysis can determine that `x` and `y` can only be null or *q*. Thus the conditional in Thread 2 is equivalent to

```
if (r2 != q) {
  construct Foo in *q;
  r2 = q;
}
```

As in RFUB Example 1, the value stored into `x` by Thread 2 is independent of program variables, and thus the store can potentially be moved, either by the compiler itself or by the hardware, up to the beginning of Thread 2. If Thread 1 again executes immediately after this store, before the rest of Thread 2, we get an execution in which `x` and `y` both end up with the value *q*, but the constructor is never executed, and thus the virtual function call branches to an uninitialized address.

We emphasize that the issue here is not visibility of the constructor's effects in another thread; the result of the new expression ends up getting used while the associated constructor is not executed *at all*.

Note that all potential accesses to the memory at *q* are by Thread 2, and ordered by sequenced-before. Many programmers would assume that it is fundamentally impossible for the value of a new-expresion to be used without execution of the constructor. This demonstrates that in the context of a data-race-free program (in the sense of the standard) using `memory_order_relaxed`, this is wrong.

Possible fixes and their cost

The kind of problems we have seen here cannot occur at the assembly language level, or with a naive compiler. Weak hardware memory models like ARM and Power allow the crucial reordering of a load followed by a store, but only if the store does not "depend" on the load. The architectural specification includes a definition of "depend". For our examples, when naively compiled, the final store "depends" on the initial load, so the reordering would be prohibited, preventing the problematic executions.

The core problem is that the architectural definitions of dependence are not preserved by any reasonable compiler. In order to preserve the intent of the hardware rules, and to prevent OOTA behavior, we need to strengthen this notion of dependence to something that can reasonably be preserved by compilation. Many attempts to do so in clever ways have failed. The RFUB examples argue that this is not possible in the context of the current specification, since the only difference between full OOTA behavior, and behavior allowed by current implementations, is an unexecuted if-branch. And even if we could make this distinction, it wouldn't be useful, since implementations currently allow behavior that we don't know how to reason about.

The cleanest general solution is to strengthen this notion of "dependence" to simply the "sequenced before" relation, and thus require that a relaxed load followed by a relaxed store cannot be reordered. This clearly has the disadvantage of disallowing current implementations. We previously proposed a variant of this solution in N3710. Any change in this direction may need to be phased in in some way, in order to allow compiler, and possibly hardware, vendors time to adjust.

This is also the solution that was formalized in http://plv.mpi-sws.org/scfix/paper.pdf. The formal treatment is quite simple; we just insist that the "reads from" relation is consistent with "sequenced before". We also expect the standards wording to be quite simple.

The performance of this approach was recently studied in Peizhao Ou and Brian Demsky, "Towards Understanding the Costs of Avoiding Out-of-Thin-Air Results" (OOPSLA'18). The overhead (on current implementations) of preventing load; store reordering on ARM or Power is much less than that of enforcing acquire/release ordering. There are some uncertainties about machine memory models in this area, but the expectation is that the required ordering can be enforced by inserting a never-taken branch or the like after `memory_order_relaxed` loads. In many cases, even this will not be needed, because an existing branch, load, or store instruction either already fulfills that role, or can be made to do so. No CPU architecture appears to require an actual expensive fence instruction to enforce load;store ordering.

I am not aware of any cost measurements for enforcing load;store ordering on a GPU. Such measurements would be extremely useful.

Pursuing this path will require some compiler changes around the compilation of `memory_order_relaxed`. And I expect it would eventually result in the introduction of better hardware support than the current "bogus branch" technique.

Participants in the external, non-WG21, discussion largely agreed that we need a replacement or replacements for `memory_order_relaxed`. But there is no clear consensus on many more detailed questions:

1. Should we simply strengthen the semantics of `memory_order_relaxed` along the above lines, or leave it as is, and add a stronger version amenable to precise reasoning?
2. It seems likely that even if we replace `memory_order_relaxed` semantics with the stronger semantics, we will want to offer one or more more specialized weaker versions, since some specific usage idioms do not require the stronger treatment. For example, a relaxed atomic increment of a counter often does not require the strengthening, because the result of the implied load is not otherwise used. Should there be one such weaker order, or one for each idiom?
3. If we have such weaker order(s), should they be specified in the same style as now, or as a set of constraints under which they imply sequentially consistent behavior, as in Sinclair, Alsop, and Adve, "Chasing Away RAts: Semantics and Evaluation for Relaxed Atomics on Heterogeneous Systems", ISCA 2017 (a.k.a. DRFrlx).
4. Or should we try to follow the path of DRFrlx, and only have `memory_order_relaxed` replacements that are specified in this way?

My opinion is that, since current implementations effectively do not follow my interpretation of the OOTA guidance in the standard, and since we want to preserve correctness of current code, while actually specifying its behavior precisely, we should change the existing semantics of `memory_order_relaxed` to guarantee load;store ordering.

We should then try to develop weaker memory orders tailored to specific idioms, and specified in a DRFrlx style, to regain any performance lost by the preceding step. I currently do not believe that we will be able to find a single weaker specification that avoids our current OOTA issues. Thus I expect the specifications to be tailored to specific idioms. It currently seems useful to expose these as different `memory_orders` to make it clear what

part of the specification the user is relying on. It also seems likely that each of these will impose somewhat different optimization constraints. And many of us are of the opinion that `memory_order_relaxed` is already usable primarily in cases that match one of a small number of reasonably well-known idioms.

I haven't been convinced that the DRFrlx approach by itself is currently viable as a replacement for `memory_order_relaxed`, both due to backwards compatibility issues, and because some of the use cases we see in Android appear to be hard to cover.

The end result here should be a specification that provides performance similar to what we have now, but that is well-defined for all the `memory_orders`, rather than the current hand-waving for `memory_order_relaxed`. It would also address OOTA issues for `memory_order_consume`, but not touch the other `memory_order_consume` issues we have been discussing separately.

Full decisions here will have to wait for some of the missing information, particularly in regard to GPU performance implications. But a preliminary sense of the committee on these issues would be useful.

San Diego and other pre-Cologne discussion

SG1 discussions in San Diego, and on the reflector, suggest that there is not a consensus for strengthening std::memory_order_relaxed semantics, but there may be one for introducing an additional memory order.

There was also discussion of wg21.link/p1239r0. My concern with this approach is that it shifts the burden of identifying and avoiding potential OOTA results to the programmer. I do not know how to program in such an environment; in particular, I do not see how to recognize and address cases such as OOTA Example 3. This issue becomes even more egregious if mis-speculated loads can visibly cause undefined behavior because they produce some sort of trap representation. I do not believe that any proposal that explicitly requires programmers to anticipate causal cycles can result in a viable programming model.

A compromise proposal

Although there is not a consensus for strengthening `memory_order_relaxed`, there does seems to be an increasing consensus about what constitutes acceptable behavior for `memory_order_relaxed`. Most participants seemed to feel that even the "RFUB New Constructor Example" should be allowed, and is not excluded by our vague OOTA prohibition.

I thus currently believe that the most promising way forward is along the following lines:

1. Let `memory_order_relaxed` keep the current semantics, as defined by current implementations. There are a number of use cases which remain correct if racing loads can return an arbitrary value. (These include simple counters that are either only read after all threads are joined, initial loads used with compare_exchange, etc.) `std::memory_order_relaxed` is (possibly aside from progress issues) provably correct for such uses, even in the presence of OOTA results, and that would not change.
2. Add a stronger std::memory_order_load_store, along the lines proposed above.
3. Add an example along the lines of "RFUB New Constructor Example" above, to make it clear that trusting the result of a racing `memory_order_relaxed` can be hazardous, and can result in very strange behavior that can cause seemingly sequential code sections to misbehave.

For clarity here is an initial attempt at wording for the second point. We ignore the issue of breaking this up appropriately between the atomics clause and [intro.races]. We also, for now, omit the wording to add memory_order::load_store and memory_order_load_store to [30.4] atomics.order.

Add an additional definition along the lines of:

> An evaluation A is load-store ordered before an evlauation B if any of the following holds:
>
> 1. A is a `memory_order_load_store`, `memory_order_release`, or `memory_order_seq_cst` store, and B is a `memory_order_load_store` load that takes its value from the store A.
> 2. A is a `memory_order_load_store` store, and B is a `memory_order_load_store`, `memory_order_acquire`, or `memory_order_seq_cst` load that takes its value from the store A.
> 3. A happens before B.
> 4. There is a an evaluation C, such that A is load-store ordered before C, and C is load-store ordered before B.

Modify 6.8.2.1p10 [intro.races] as follows:

> An evaluation A happens before an evaluation B (or, equivalently, B happens after A) if:
>
> - (10.1) A is sequenced before B, or
> - (10.2) A inter-thread happens before B.
>
> The implementation shall ensure that no program execution demonstrates a cycle in the ~~“happens before” relation. [Note: This cycle would otherwise be possible only through the use of consume operations. — end note]~~ "load-store ordered before" relation. [Note: Such cycles could arise if a `memory_order_load_store` store were visibly reordered witn a `memory_order_load_store` store operation sequenced after it. This also constrains the implementation of `memory_order_consume` operations. --end note]